Industrial Vision

Lecture 2 - Spatial Filtering

ID6004W

+ board

Where we are

- In the previous lecture we showed that **linearity and shift invariance together imply convolution**. Two examples were the **box** filter, whose weights sum to 1 and which smooths, and the **gradient** filter, whose weights sum to 0 and which detects change.
- Gradient filters are **first** derivatives. Today we introduce the **second** derivative, the **Laplacian**, and show that it requires the **Gaussian** if it is to be robust to noise.

+ board

Second derivatives in space

the Laplacian · zero-crossings · LoG and DoG · denoising · sharpening

+ board

The Laplacian as a second derivative

The Laplacian is the sum of the unmixed second partial derivatives. As a **continuous** operator it is **isotropic**, meaning that it has no preferred direction:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

We discretize it using a **Taylor series**. For a 1D slice we expand about both neighbours:

$$f(x+h) = f(x) + hf'(x) + \tfrac{h^2}{2} f''(x) + \cdots$$
$$f(x-h) = f(x) - hf'(x) + \tfrac{h^2}{2} f''(x) - \cdots$$

Adding the two expansions cancels the odd-order terms, which are the ones containing the first derivative:

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + \tfrac{h^4}{12} f''''(x) + \cdots \implies f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

It is the discarding of the h^4 term that makes this an approximation, and that makes the stencil **second-order accurate**. On the pixel grid $h = 1$, which gives the weights $\begin{bmatrix} 1 & -2 & 1 \end{bmatrix}$.

+ board

Assembling the 2D kernel

The second derivative in x is a horizontal stencil, and the second derivative in y is its transpose:

$$\partial_{xx} : \begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \qquad \partial_{yy} : \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

We add them to obtain $\nabla^2 = \partial_{xx} + \partial_{yy}$:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \qquad \text{the entries sum to } 0$$

The Laplacian is of rank two and accordingly splits into **two** separable passes:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & -2 & 1 \end{bmatrix}$$

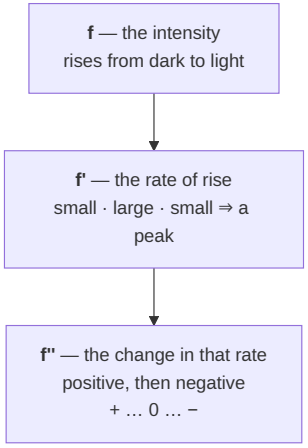
which is precisely $\partial^2/\partial x^2 + \partial^2/\partial y^2$, one separable term for each second derivative.

+ board

Zero-crossings locate edges

A real edge is a **ramp** rather than an abrupt jump, since the intensity takes several pixels to rise. We must therefore decide which pixel on that ramp is to be called the edge.

Consider the three functions along a line crossing the edge:



- The intensity rises **most rapidly at the middle of the ramp**, and it is that steepest point which we take to be the edge. The two derivatives are simply two ways of identifying it.
- Before the middle the rate of rise is **increasing**, so $f'' > 0$; after it the rate is **decreasing**, so $f'' < 0$. In passing from positive to negative, f'' must take the value **zero**.
- The zero of f'' therefore occurs at **precisely** the position at which $|f'|$ attains its peak. The same point is marked twice: as a **maximum** of the first derivative, and as a **change of sign** of the second. The change of sign is the marker that can be located accurately, for the reason given on the next slide.

Crossing zero is not the same as being zero. In a region of constant intensity $f'' = 0$ everywhere, and there is no edge. What is required is a **change of sign**, not a small value; and since a small ripple can also change the sign, a crossing is accepted only where $|f'|$ is large.

+ board

Zero-crossings in Python - the 1D case

We place a ramp edge at $x_0 = 63.4$, which is deliberately **not** an integer, and ask each derivative to recover its position.

```
x = np.arange(128.0)
f = 128 + 90*np.tanh((x - 63.4)/6)   # ramp edge

d1 = np.gradient(f)                  # f'
d2 = f[2:] - 2*f[1:-1] + f[:-2]      # f'', [1 -2 1]

peak = np.argmax(abs(d1))            # -> 63  (on-grid)

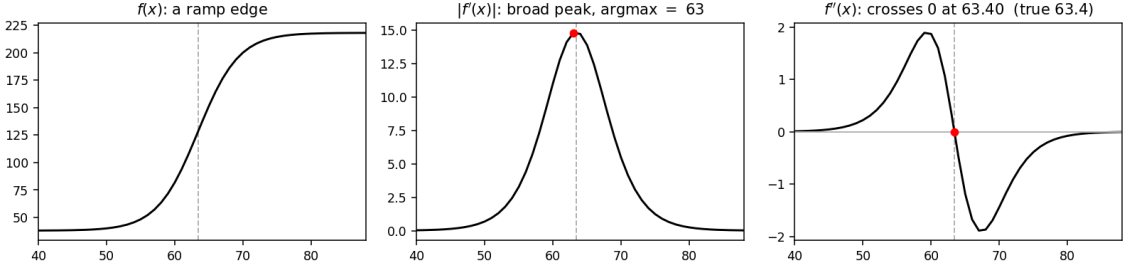
i  = np.flatnonzero(np.sign(d2[:-1]) != np.sign(d2[1:]))[0]
t  = d2[i] / (d2[i] - d2[i+1])       # linear interp.
zc = (i + 1) + t                     # -> 63.402
```

- `argmax` can only return an **integer index**. It reports 63, whereas the true position is 63.4. The error is 0.4 px here, and can be no smaller than 0.5 px in the worst case.
- The crossing of f'' lies **between** two samples, and we may ask where between them it falls. Joining the two values that straddle zero by a straight line and solving for the intercept gives $t = \dfrac{f''_i}{f''_i - f''_{i+1}}$, hence 63.402, an error of 0.002 px.

Why the peak does not permit this. The computed values at $x = 62, 63, 64$ are

$$|f'| \;:\; 14.10,\quad \mathbf{14.80},\quad 14.72$$
$$f'' \;:\; +1.07,\quad \mathbf{+0.33},\quad \mathbf{-0.49}$$

A maximum is **flat at its summit**: the two largest samples differ by only 0.08, so a small amount of noise is enough to exchange them. A crossing is **steep**: the two straddling samples differ by 0.81, ten times as much. A flat function cannot be interpolated usefully; a steep one can.

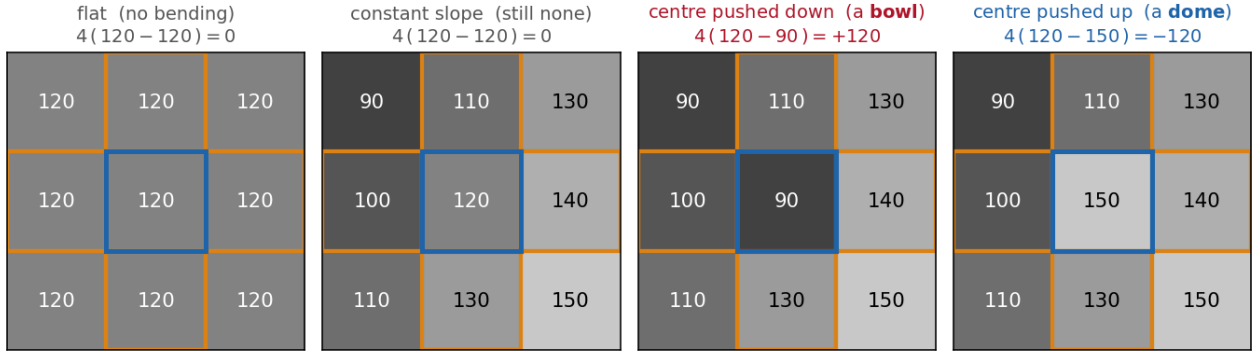


In 2D, ∇^2 asks one question at every pixel

Grouping the terms differently, the kernel ceases to be a table of weights and becomes a **question about the centre pixel**:

$$\nabla^2 f[m,n] \;=\; \underbrace{f[m{-}1,n] + f[m{+}1,n] + f[m,n{-}1] + f[m,n{+}1]}_{\text{the four neighbours}} \;-\; 4\,f[m,n] \;=\; 4\Big(\overline{f}_{\text{nbrs}} \;-\; f[m,n]\Big)$$

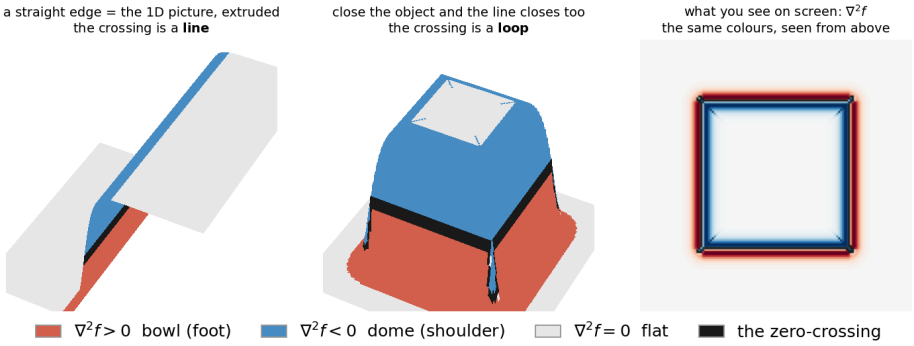
*Is this pixel **above** or **below** the average of its four neighbours?*



If the centre is **darker** than its neighbours the surface dips there, forming a bowl, and $\nabla^2 f > 0$. If it is **brighter** the surface bulges, forming a dome, and $\nabla^2 f < 0$. If the values agree the response is 0. Patches 2, 3 and 4 share the same background and differ **only in the centre pixel**, and the responses are $0, +120$ and -120. The operator measures **the departure of the centre from its neighbourhood**.

Intensity read as height: an edge is a hillside

Take the grey value to be a **height**. Since ∇^2 measures **bending**, a path leading from a valley up onto a plateau must bend **twice, in opposite directions**.



Following such a path, the colours below are the sign of $\nabla^2 f$ and mean the same thing in all three panels:

- On the **valley floor** and on the **plateau** the surface is flat and nothing bends: $\nabla^2 f = 0$.
- At the **foot** of the slope the surface curves **upward** out of the flat, forming a **bowl**: $\nabla^2 f > 0$.
- At the **shoulder** it curves **over** onto the plateau, forming a **dome**: $\nabla^2 f < 0$.
- Between the bowl and the dome the bending must pass through **zero**. That seam, drawn in black, **is the edge**, and it lies at the **steepest** point of the slope, where $|\nabla f|$ attains its peak.

A single edge therefore produces **two bands of opposite sign**. The edge is not to be sought within either band; it is the **boundary between them**.

A straight edge is the one-dimensional case extruded. Let t run along the edge and n across it. Nothing varies along a straight edge, so $\partial^2 f/\partial t^2 = 0$ and

$$\nabla^2 f \;=\; \frac{\partial^2 f}{\partial t^2} + \frac{\partial^2 f}{\partial n^2} \;=\; \frac{\partial^2 f}{\partial n^2}$$

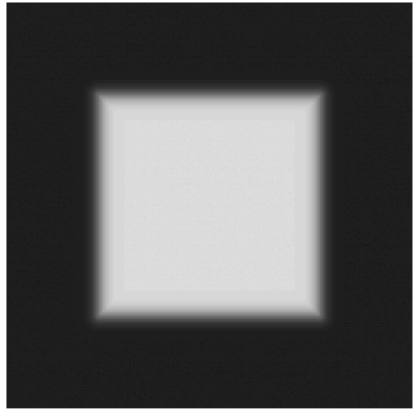
Nothing new therefore occurs: the operator reduces to the one-dimensional second derivative taken across the edge and repeated at every t. The crossing **point** of the previous slide, carried along the edge, becomes a crossing **line**; and if the object is closed, the line closes with it, as in panel 2.

Corners are the exception. There *both* second derivatives contribute at once, which is why the corners in panel 3 are about 8 times stronger than the straight sides, 62.8 against 7.5 for this square.

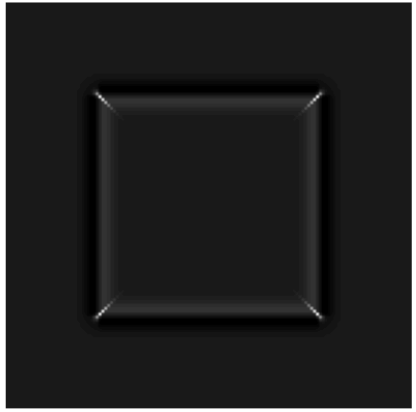
+ board

On the grid: the crossing lives **between** the pixels

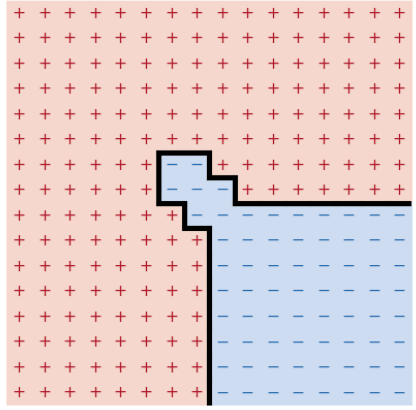
the original f



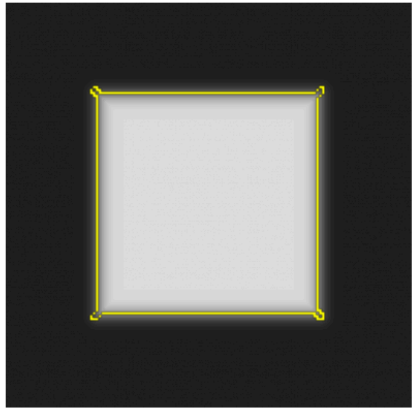
the actual Laplacian $\nabla^2 f$ the min/max value=-62 / 7



zoomed on a corner: the crossing runs *between* the pixels, and never breaks



the same curve laid back over f: it traces the boundary



+ board

Differentiation amplifies noise, and by a calculable amount

Let the observed image be $g = f + \eta$, with η independent and identically distributed and $\mathrm{Var}[\eta] = \sigma_\eta^2$. A first difference subtracts **two independent** noise samples, and independent variances **add**:

$$g(m{+}1,n) - g(m{-}1,n) \;=\; \underbrace{f(m{+}1,n) - f(m{-}1,n)}_{\text{signal}} \;+\; \underbrace{\eta(m{+}1,n) - \eta(m{-}1,n)}_{\text{noise}}, \qquad \mathrm{Var} = \sigma_\eta^2 + \sigma_\eta^2 = 2\sigma_\eta^2$$

The same argument holds for **any** linear filter w. The output noise is $\sum_k w_k\, \eta_k$, a weighted sum of independent variables, so that

$$\mathrm{Var}\left[\, w * \eta \,\right] \;=\; \sigma_\eta^2 \sum_k w_k^2$$

$\sum_k w_k^2$ is the *noise gain* of the filter

filter	weights	$\sum w^2$	output noise
box 3×3 (average)	$\frac{1}{9}$ each	0.111	$0.33\,\sigma_\eta$
gradient ∂_x	$[\,1 \;\; 0 \;\; -1\,]$	**2**	$1.41\,\sigma_\eta$
second difference ∂_{xx}	$[\,1 \;\; -2 \;\; 1\,]$	6	$2.45\,\sigma_\eta$
Laplacian ∇^2 (5-point)	$1, 1, 1, 1, -4$	**20**	$\mathbf{4.47}\,\sigma_\eta$

An average divides the noise, whereas a difference multiplies it. A set of weights summing to 1 may have $\sum w^2 \ll 1$; a set summing to 0 cannot, since the weights must be large enough to cancel one another.

The gradient costs a factor $\sqrt{2}$ and the Laplacian a factor $\sqrt{20} = 4.5$, of which the centre weight -4 alone contributes 16 of the 20. **This is why the unsmoothed ∇^2 image is dominated by noise.** One noise field, three operators, and **a single grey scale**:



η: $\sigma_{\text{out}} = 7.97$ $\partial_x \eta$: 11.24 $\nabla^2 \eta$: 35.54

+ board

Why do the variances add?

Suppose

$$X = \eta(m+1, n), \qquad Y = \eta(m-1, n).$$

The noise is assumed **independent**, so

$$\mathrm{Cov}(X, Y) = 0.$$

The variance of a difference is

$$\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) - 2\,\mathrm{Cov}(X, Y).$$

Since the covariance is zero,

$$\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y).$$

Each has variance σ_η^2, so

$$\mathrm{Var}(X - Y) = 2\sigma_\eta^2.$$

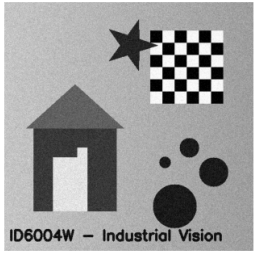
Notice that **subtracting** random variables does **not** subtract their variances. Variance measures spread, and the spreads combine.

+ board

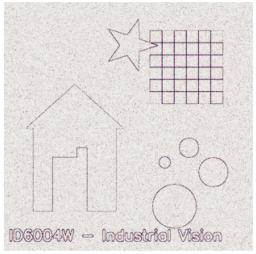
Smoothing first: the gain, in numbers

Suppose we blur with G_σ and differentiate afterwards. The two steps compose into a **single** filter $w = \partial * G_\sigma$, so the rule above applies unchanged: convolve the two kernels and sum the squares of the result.

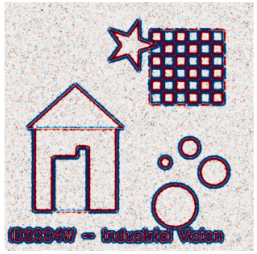
$g = f + \eta,\ \sigma_\eta = 12$



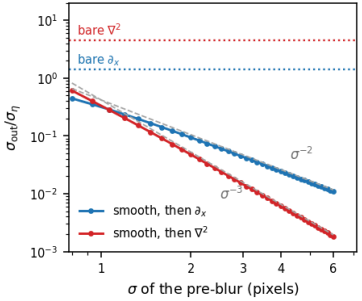
$\nabla^2 g$ straight away
noise floor $\sigma_{\text{out}} = 53.67$ (shown at $\pm 6\sigma_{\text{out}}$)



smooth first, then ∇^2 $(\sigma = 2)$
noise floor $\sigma_{\text{out}} = 0.57$ (shown at $\pm 6\sigma_{\text{out}}$)



noise gain vs σ, measured



σ	$\sum w^2, \partial_x * G_\sigma$	vs. bare 2	$\sum w^2, \nabla^2 * G_\sigma$	vs. bare 20
1	0.101	20× quieter	0.112	179×
2	0.0088	227×	0.00227	8 800×
4	0.00060	3 300×	0.000038	530 000×

Asymptotically $\sum w^2 \to \frac{1}{2\pi\sigma^4}$ for the gradient and $\frac{1}{2\pi\sigma^6}$ for the Laplacian. **Doubling σ therefore divides the noise variance of the gradient by 16 and that of the Laplacian by 64**, which are the slopes -2 and -3 in panel 4.

Smoothing weakens the signal as well, so a ratio of variances alone overstates the benefit. The appropriate measure on a step edge of height A is $\text{SNR} = (\text{peak response})/\sigma_{\text{out}}$:

operator	peak	σ_{out}	SNR
bare ∂_x	A	1.41	0.71
$G_2 * \partial_x$	$0.38A$	0.094	**4.00**
bare ∇^2	A	4.47	0.22
$G_2 * \nabla^2$	$0.056A$	0.048	**1.18**

The improvements, 5.7 and 5.3 times respectively, remain substantial. Measured over a range of σ, the SNR of a straight edge grows **linearly in σ**.

There is a price. σ is a **scale**, and detail finer than σ is removed along with the noise. **By linearity the two steps form a single operator, which is the subject of the next slide.**

+ board

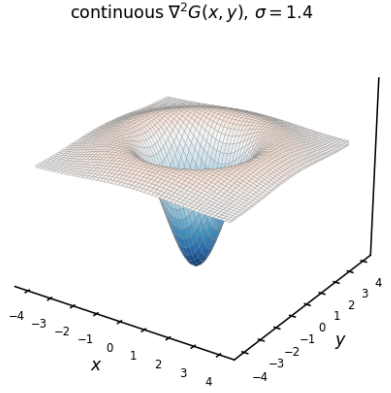
Smoothing before differentiation: the LoG

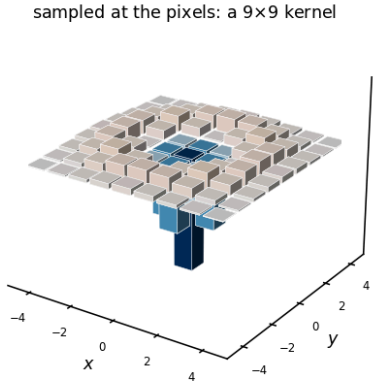
The cost of each step is now known: the unsmoothed ∇^2 multiplies the noise by $\sqrt{20}$, while a preliminary blur reduces it by a factor of order a thousand. We therefore **smooth first and differentiate afterwards**, and by linearity the two operations constitute a **single** operator:

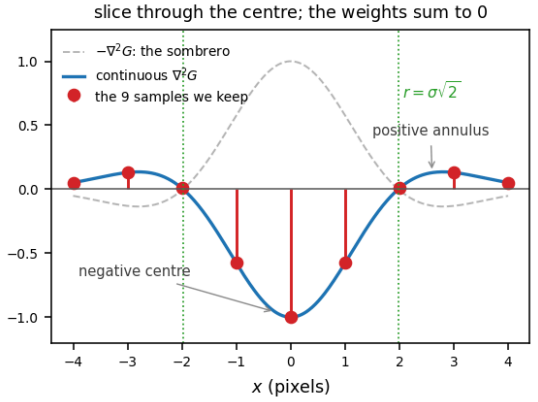
$$\nabla^2(G * f) \;=\; (\nabla^2 G) * f \qquad \Rightarrow \qquad \mathrm{LoG} = \nabla^2 G$$

The LoG is a **function** of x and y, and the **filter** is that same function evaluated at the integer pixel positions.

$$\nabla^2 G(x,y) = \frac{x^2+y^2-2\sigma^2}{\sigma^4}\, e^{-\frac{x^2+y^2}{2\sigma^2}} \qquad \Longrightarrow \qquad w[m,n] = \nabla^2 G(m,n), \quad -r \le m,n \le r$$



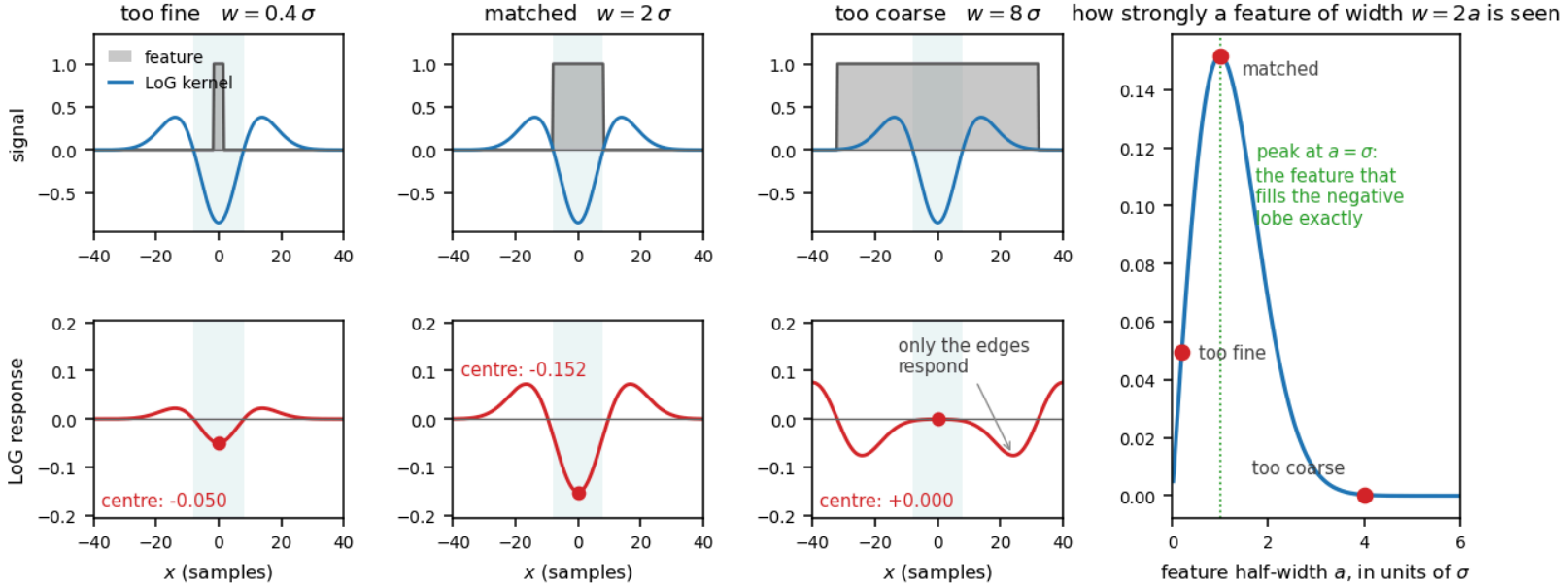




What σ selects, in 1D, where the result is exact

We now fix the filter and vary the signal. In one dimension the operator is G'', whose negative lobe spans $|x| < \sigma$. A bar of half-width a produces, **at its own centre**, exactly

$$(G'' * \mathrm{bar})(0) \;=\; \int_{-a}^{a} G''(t)\,\mathrm{d}t \;=\; 2\,G'(a) \;=\; -\frac{2a}{\sigma^2}\,e^{-a^2/2\sigma^2} \qquad \text{maximal at} \quad a = \sigma$$



In one dimension the sign change occurs at $|x| = \sigma$. In two dimensions the Laplacian adds two second derivatives, which moves it outwards to $r = \sigma\sqrt{2}$.

LoG

A single operator performs both tasks: it smooths with a Gaussian **and** applies the Laplacian. Its shape, a negative well inside a positive ring, is the one known as the **Mexican hat**.

What does σ select? The mask is negative at the centre and positive in a surrounding annulus, changes sign at $r = \sigma\sqrt{2}$, and has weights summing to zero. It therefore compares a pixel with a **ring** of neighbours lying at a distance of order σ.

- Detail **finer** than σ is smoothed away before the differentiation reaches it.
- Detail **coarser** than σ is very nearly constant across the whole mask, and the zero sum removes it.

The operator thus responds only to a **band** of scales near σ, and is neither a smoothing filter nor a plain differentiator. Stating this precisely requires a language for scale, which the second half of the lecture provides.

Reading the shape. There is a deep negative well, a sign change on the circle $r = \sigma\sqrt{2}$, and then a shallow positive annulus decaying to zero. The crest of the annulus lies at $r = 2\sigma$, and the well is exactly $e^2 \approx 7.4$ times deeper; the annulus nevertheless contains many more pixels, and the two contributions cancel, so that $\sum_{m,n} w[m,n] = 0$. The overall sign is a matter of convention. Texts that draw the hat the other way up are plotting $-\nabla^2 G$, and the position of a zero-crossing is unaffected.

The cost of sampling. The tail must be retained. If the kernel is truncated at $\pm 2.9\sigma$, the weights still sum to 15% of the centre weight, so the filter no longer ignores a constant region. One should take $r \gtrsim 4\sigma$ and then **subtract the mean**, which restores the zero sum exactly.

At the opposite extreme, below about $\sigma = 1$ the hat no longer fits on the grid, since the ring falls between samples. The support collapses to 3×3 and the operator reduces to the bare Laplacian with which we began. This is the limit $\nabla^2 G \to \nabla^2 \delta$ as $\sigma \to 0$, and it is why the LoG is useful only for σ of a pixel or more.

+ board

Zero-crossings in Python - the 2D case

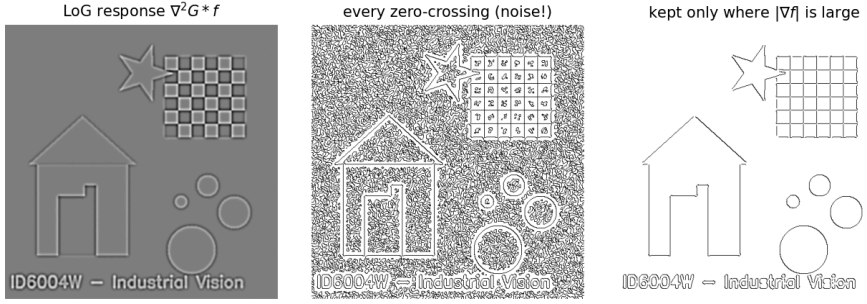
In two dimensions a zero-crossing is a **change of sign between neighbours** of the LoG response, which is then gated on the gradient magnitude:

```
log = cv2.filter2D(f, cv2.CV_64F, log_kernel(sigma=2))
s   = np.sign(log)

zc = np.zeros(log.shape, bool)       # sign change against
zc[:, :-1] |= s[:, :-1] != s[:, 1:]  #   the right neighbour
zc[:-1, :] |= s[:-1, :] != s[1:, :]  #   the lower neighbour
# 71966 pixels -- most of them in flat regions!

gx, gy = cv2.Sobel(...), cv2.Sobel(...)
mag    = np.hypot(gx, gy)
edges  = zc & (mag > np.percentile(mag, 90))   # 5284
```

- In a flat region $\nabla^2 G * f$ is zero only up to the noise, so its **sign is determined by the noise** and neighbours disagree continually. Each such disagreement is a zero-crossing.
- Gating on $|\nabla f|$ discards **93%** of them and retains the object boundaries.
- The **ungated** crossings form **closed** contours, since a change of sign cannot simply stop, whereas the gradient-peak edges of L03 carry no such guarantee. There is a trade-off, however: **gating is precisely what opens these contours**, because it cuts them wherever the gradient is weak. Closure and selectivity therefore work against each other.



Averaging as a means of denoising

Suppose we capture L frames of a static scene in quick succession, so that $g_i = f + \eta_i$ with $\eta_i \sim \mathcal{N}(0, \sigma_\eta^2)$ independent and identically distributed. Here σ_η is a **noise** level, not the width of a Gaussian kernel. We average the frames:

$$\hat{f} = \frac{1}{L} \sum_{i=1}^{L} g_i$$

The estimator is **unbiased**, that is, correct on average:

$$\mathbb{E}[\hat{f}] = f$$

It is also **consistent**, since the noise decreases as more frames are used:

$$\mathrm{Var}(\hat{f}) = \frac{\sigma_\eta^2}{L}$$

If only one image is available, the averaging is performed **spatially**, and the neighbours within a local window take the place of the other frames.

There is a difficulty. Across an **edge** the neighbours do not share a common value, so averaging there **smears** the edge. It is this blurring that makes edge-preserving filters necessary later.

+ board

The median filter, for cases in which averaging fails

Salt-and-pepper noise consists of isolated black and white specks. It defeats linear filters, because a single outlier displaces the average. We therefore use an **order statistic** instead.

The median filter sorts the values in the window, takes the **middle** value, and writes it to a new array.

An outlier is placed at one end of the sorted list and is therefore ignored by the median, whereas a genuine edge, which appears as a run of similar values, is preserved.

The filter is **non-linear**, which has two consequences.

- There is no kernel, so **superposition does not hold** and the FFT offers no shortcut.
- A **weighted median** repeats selected values before sorting, which emphasizes them without performing a convolution.

+ board

Unsharp masking: sharpening by subtracting a blurred copy

The name is misleading. The image is **smoothed**, which is the unsharp step, and the result is then **subtracted**, which is the masking step, so as to enhance the detail.

$$I_{\text{sharp}} = I + k\,(I - I_b), \qquad I_b = I * G, \qquad k \in [0.2,\ 0.7]$$

- The difference $I - I_b$ is the **high-frequency detail**, such as edges and texture, that the blur has removed.
- Adding a scaled copy of that detail back to the image produces sharper edges.

A useful identity. Since $I_b = I * G$, we have

$$I - I_b = I * (\delta - G)$$

The detail term is therefore the output of a single filter with kernel $\delta - G$, which **passes the whole image and then removes the part retained by the blur**.

References

1. D. A. Forsyth and J. Ponce, *Industrial Vision: A Modern Approach*, 2nd ed. Pearson, 2012. See **Ch. 4** on linear filters, Gaussians and sampling.
2. D. Marr and E. Hildreth, "Theory of edge detection," *Proc. R. Soc. Lond. B*, vol. 207, no. 1167, pp. 187-217, 1980. This paper introduces the LoG and zero-crossings.

+ board